

Scalable Plagiarism Detection via Hashing, Dynamic Programming, and Parallel Processing

JAKIA NOWSHIN AND SURAYIA RAHMAN

DS8001 FINAL PROJECT
Fall 2025

December 12, 2025

Contents

1	Introduction	4
2	Literature Review	4
2.1	Classical Algorithmic Paradigms	5
2.1.1	Scalable Hashing and Lexical Overlap	5
2.1.2	Dynamic Programming and Sequential Structure (LCS)	5
2.1.3	Segmented Set-Based Matching and Fragment-Level Similarity	6
2.2	Modern Semantic Paradigms	6
2.2.1	Sentence Embeddings and Semantic Similarity	6
2.2.2	Architectural Refinement: MPNet and Euclidean Distance	7
3	Methodology	7
3.1	Obfuscation-Based Algorithmic Synthesis	7
3.1.1	Dataset: PAN-PC11 Artificial Plagiarism Corpus	7
3.1.2	MinHash + Locality-Sensitive Hashing (LSH)	8
3.1.3	Longest Common Subsequence (LCS) with Porter Stemming	10
3.1.4	Divide & Conquer Plagiarism Detection with Jaccard Similarity	11
3.1.5	LLM Approach	13
3.2	Type-Agnostic Semantic Refinement (TASR)	14
3.2.1	Data Preprocessing	14
3.2.2	Alignment Between Ground Truth Features and Code Implementation	18
3.2.3	Algorithmic Approach (Structural Phase)	19
	(a) Configuration File (config.py)	19

(b) Index Construction via MinHashing	20
(c) Detection and Evaluation	22
(d) Aggressive Pipeline	23
(e) Analyzing XML Features	24
(f) DP-based LCS Insight	25
3.2.4 LLM pipeline	25
(a) Consolidated LLM Data	25
(b) Establishing LLM Detection	26
(c) Semantic Embedding Index	27
(d) Evaluation of the Model	28
4 Experimental setup	30
4.1 Obfuscation-Based Algorithmic Synthesis	31
4.2 Type-Agnostic Semantic Refinement	31
4.2.1 Algorithmic Baseline	32
4.2.2 Semantic Baseline and Diagnosis	32
4.2.3 Rationale for Architectural Upgrade	33
5 Results and Discussion	33
5.1 Obfuscation-Based Algorithmic Synthesis	35
5.2 Type-Agnostic Semantic Refinement	35
5.2.1 Algorithmic Baseline Results	35
5.2.2 Aggressive pipeline	38
5.2.3 LLM approach	42
6 Conclusion	44

6.1	Project Insights and Weaknesses	45
6.2	Future Research Directions	45

1 Introduction

Plagiarism detection is an algorithmic challenge, particularly as paraphrasing techniques increasingly obscure lexical overlap and disrupt structural cues. The PAN-PC11 corpus provides a controlled benchmark with source-suspicious document pairs annotated by ground-truth obfuscation levels, enabling precise evaluation of algorithmic robustness under artificially generated plagiarism.

Our system is evaluated using the PAN Plagiarism Corpus 2011 (PAN-PC-11),⁸ a widely adopted dataset in the field of plagiarism detection research.

This study first evaluates classical, structural, and token-based methods. **MinHash-LSH** captures global lexical overlap efficiently, the **Longest Common Subsequence (LCS)** algorithm evaluates sequential similarity, and **Divide & Conquer (D&C) with Jaccard similarity** recursively compares document segments to detect localized copying. While these approaches perform well under low obfuscation, recall declines as paraphrasing intensifies. A lightweight LLM-based method (**SBERT**) is included primarily for comparative insight, demonstrating how semantic embeddings can partially recover meaning when structural signals collapse.

Building on this analysis, the **Type-Agnostic Semantic Refinement (TASR)** methodology targets artificial plagiarism cases identified in the PAN-PC11 ground-truth metadata. TASR employs a preprocessing pipeline where all 11,093 source documents are globally indexed, the corpus is recursively scanned, and XML metadata is parsed to extract plagiarism type. Structural signal degradation is assessed through MinHash-LSH and LCS, a semantic baseline is established with MiniLM embeddings, and the architecture is refined with higher-capacity MPNet embeddings and Euclidean similarity to improve detection under extreme paraphrasing.

Together, these evaluations provide an **algorithm-centered understanding** of plagiarism detection, illustrating how structural hashing, sequence alignment, recursive segment comparison, and targeted semantic refinement can be combined to detect artificially obfuscated text, with LLM embeddings serving primarily as a comparative benchmark.

2 Literature Review

Plagiarism detection research has traditionally been divided into two dominant methodological paradigms: structural/lexical analysis and semantic/conceptual modeling. The present study examines these paradigms to evaluate their performance limits, using the *Artificial* plagiarism subset of the PAN-PC11 Corpus⁵—a benchmark widely recognized as the most challenging test case for extrinsic plagiarism detection due to its deliberate high obfuscation.

2.1 Classical Algorithmic Paradigms

2.1.1 Scalable Hashing and Lexical Overlap

Locality-Sensitive Hashing (LSH) and its underlying probabilistic foundation, MinHash, form the basis of many scalable near-duplicate detection systems. The seminal work of Broder et al.¹ formalized MinHash as a means of approximating Jaccard similarity through compact signature vectors, enabling efficient large-scale retrieval. Subsequent research has highlighted both the strengths and limitations of this paradigm. For example, Hoad and Zobel² demonstrate that although MinHash and related hashing strategies provide excellent performance for exact or lightly modified duplicates, they are intrinsically brittle under paraphrasing, synonym substitution, sentence reordering, and other structural manipulations.

In alignment with these findings, the OBAS experiments in the present study empirically confirmed this vulnerability. Even under aggressive configurations designed to maximize recall, the LSH-based approach was unable to detect plagiarized text when the underlying lexical and sequential structure had been disrupted. This outcome mirrors the broader consensus in the literature: structural algorithms cannot maintain accuracy when lexical continuity is intentionally destroyed.

2.1.2 Dynamic Programming and Sequential Structure (LCS)

Dynamic Programming (DP)–based methods, exemplified by the Longest Common Subsequence (LCS) algorithm developed by Needleman and Wunsch,³ provide a more rigorous measure of sequential overlap by explicitly quantifying preserved subsequences. LCS and related DP-based similarity metrics have been widely applied in text alignment tasks and are frequently enhanced through preprocessing methods such as stemming⁴ to normalize lexical variants.

However, the DP paradigm suffers from two well-documented limitations:

1. computational complexity: LCS runs in $O(m \times n)$, limiting scalability for long documents, and
2. structural fragility: even moderate reordering or paraphrasing breaks sequence continuity, causing the similarity signal to collapse.

Consistent with prior research, the LCS experiments within the TASR methodology demonstrated that average structural preservation in the Artificial subset was approximately 35%, far below the threshold required for meaningful retrieval. This finding provided strong empirical and mathematical justification for abandoning all sequence-dependent methods.

2.1.3 Segmented Set-Based Matching and Fragment-Level Similarity

A third classical paradigm in plagiarism detection is fragment-based comparison, which addresses the limitations of hashing and DP methods by evaluating similarity at smaller, localized scales. Early work on passage-level similarity (e.g., Shivakumar & Garcia-Molina, 1995; Lyon et al., 2001) showed that dividing documents into smaller units enables detectors to capture partial or region-specific copying even when global similarity is weak.

Divide & Conquer methods extend this principle by recursively segmenting documents and comparing each pair of subregions using set-based measures such as Jaccard similarity. This approach is motivated by two observations: (1) plagiarism is often confined to small fragments rather than entire documents, and (2) set-based similarity tolerates certain structural changes better than sequence-based algorithms, though it remains sensitive to deep lexical obfuscation.

The D&C strategy implemented in this study follows these principles. By evaluating similarity at multiple granularities, it detects localized plagiarism that hashing may overlook due to bucketization and that LCS may miss due to broken sequence structure. However, as noted widely in the literature, its effectiveness declines sharply under high obfuscation, where substantial vocabulary changes suppress the Jaccard signal.

2.2 Modern Semantic Paradigms

2.2.1 Sentence Embeddings and Semantic Similarity

The inability of structural techniques to detect meaning-preserving transformations has driven the field toward semantic modeling based on Transformer architectures.⁷ Sentence-BERT (SBERT)⁶ represents a key advancement by enabling sentence-level embeddings that capture conceptual similarity robustly, even under extensive paraphrasing.

However, the initial semantic baseline in this study—using the 384-dimensional MiniLM encoder—exposed two critical limitations also noted in recent literature:

- Low vector dimensionality reduces the semantic resolution of the embedding space, increasing the risk of source dilution and lowering precision.
- Cosine similarity, while widely adopted, becomes suboptimal for high-obfuscation conditions where vector dispersion increases, reducing recall.

These structural weaknesses within the embedding space itself highlighted the need for a refined semantic architecture.

2.2.2 Architectural Refinement: MPNet and Euclidean Distance

Building upon this diagnosis, the TASR methodology contributes a systematic semantic optimization rather than a generic adoption of LLMs. The refined architecture upgrades from MiniLM to the 768-dimensional MPNet encoder, providing significantly higher representational capacity. Additionally, replacing cosine similarity with Euclidean distance aligns with emerging research suggesting that Euclidean proximity more accurately reflects semantic relatedness in large, highly distributed embedding spaces.

This refinement—although computationally costly—constitutes a dedicated methodological contribution by demonstrating how embedding resolution and metric selection jointly influence detection performance in high-obfuscation environments. The resulting architecture offers a viable pathway toward achieving high F1 performance on the most resistant plagiarism category within the PAN-PC11 benchmark.

3 Methodology

3.1 Obfuscation-Based Algorithmic Synthesis

To evaluate plagiarism detection performance across different algorithmic approaches, three complementary methods were implemented: MinHash + LSH (token-overlap based), LCS (sequence-similarity based), and a Divide & Conquer segmentation method (segment-level Jaccard similarity). All methods were applied to the same PAN-PC11 document pairs, using source documents from Parts 1–10 to ensure consistent evaluation. This setup enables a controlled comparison of how different algorithmic strategies respond to varying obfuscation levels while holding the dataset, ground-truth labels, and evaluation protocol constant.

3.1.1 Dataset: PAN-PC11 Artificial Plagiarism Corpus

All three algorithms were evaluated using the PAN-PC11 artificial plagiarism dataset, a standard benchmark in plagiarism detection research. Each test case contains:

- A source document
- A suspicious document
- A predefined obfuscation level:
 - None – verbatim / near-verbatim copying
 - Low – minor editing, word substitutions

- High – heavy paraphrasing
- Unknown – unspecified transformation

Table 1: Obfuscation Levels in PAN-PC11 Dataset

Obfuscation Level	Description	How It Was Observed in the Dataset	Typical Characteristics
None	No changes applied to the source text.	The suspicious document is nearly identical to the source document.	Same wording, same structure, high token overlap, minimal variation.
Low	Minor surface-level modifications.	Slight paraphrasing or token substitutions detected when comparing source vs. suspicious documents.	Synonym replacement, small wording changes, unchanged sentence order.
High	Heavy paraphrasing or structural changes.	Significant difference in wording and structure between documents, but content still semantically related.	Sentence reordering, rephrased sentences, added/removed minor details.
Unknown	Obfuscation not labeled by dataset creators.	No explicit category provided—level inferred indirectly through similarity metrics.	Could fall anywhere on the scale depending on document pair.

In this section, only these ground-truth obfuscation levels are used to categorize and evaluate detection performance. Source documents from Parts 1–10 of the corpus are included, and results are reported strictly according to these labeled obfuscation classes. Only the detection mechanism differs.

3.1.2 MinHash + Locality-Sensitive Hashing (LSH)

(a) Preprocessing Document preprocessing is intentionally kept minimal to preserve raw token overlap, which is crucial for MinHash-based similarity:

- Convert text to lowercase
- Split text into whitespace-separated tokens
- No stemming, lemmatization, or punctuation removal

This ensures that MinHash reflects the natural token distribution of each document.

(b) MinHash Signature Generation For every document, a MinHash signature is constructed:

- `NUM_PERM = 128` hash permutations
- Each token is hashed and fed into a MinHash object
- The resulting signature approximates the document's Jaccard similarity

Two documents with more overlapping tokens have more similar MinHash signatures.

(c) Building the LSH Index (Source Documents) All source documents from parts 1–10 of the dataset are loaded and indexed into an LSH table:

- LSH uses a similarity threshold of 0.01
- Each MinHash signature is inserted with its filename as the key
- This allows fast sub-linear retrieval of candidate documents that are likely similar

The LSH index acts as a filter: only documents with sufficiently similar MinHash signatures are returned as candidates.

(d) Targeted Evaluation on Ground-Truth Pairs The evaluation is conducted on a curated set of labeled test cases, provided in `ground_truth_analysis.json`. Each test case consists of a suspicious document, its corresponding true source document, and the associated obfuscation level, which can be *none*, *low*, *high*, or *unknown*.

For each suspicious document, the evaluation proceeds as follows. The content of the suspicious document is first read and preprocessed, after which a MinHash signature is generated. This signature is then used to query the LSH index, retrieving a set of candidate source documents that share at least one bucket. The retrieved candidates are compared against the true source document to determine whether the correct source is identified. In addition, the exact Jaccard similarity between the suspicious document and its true source is computed, providing a quantitative measure of overlap. The results of this detection process are recorded and categorized according to the obfuscation level of each case.

The evaluation generates several key outputs, including recall metrics for each obfuscation level, overall recall across all test cases, execution time, and detailed detection results for every suspicious document. All results are stored in `minhash_lsh_results.json`, facilitating further analysis and performance assessment.

3.1.3 Longest Common Subsequence (LCS) with Porter Stemming

(a) Preprocessing Each document is preprocessed to retain the essential token sequence while minimizing superficial variations. The preprocessing steps include:

- Converting text to lowercase
- Tokenization by whitespace
- Stemming each token using the Porter Stemmer
- Limiting the text to the first 2,000 tokens per document

These steps ensure that the similarity computation focuses on the sequential structure of the text rather than superficial lexical variations.

(b) Method Overview This method targets sequence similarity rather than simple token overlap. For each suspicious-source document pair, the following steps are performed:

1. Compute the Longest Common Subsequence (LCS) between the stemmed token sequences using a memory-optimized dynamic programming algorithm.
2. Normalize the similarity score using

$$\text{similarity} = \frac{\text{LCS}(A, B)}{(|A| + |B|)/2}$$

where $|A|$ and $|B|$ denote the number of tokens in the suspicious and source documents, respectively.

3. Flag the pair as plagiarized if the similarity exceeds a threshold of 0.15 (15%).

(c) Evaluation Procedure For each suspicious document, the procedure involves reading the file, generating the stemmed token sequence, computing the LCS similarity with the corresponding source document, and determining detection based on the 0.15 threshold. Detection results are recorded along with execution time and obfuscation type.

(d) Performance Metrics The evaluation outputs include:

- Total detections by LCS, with reference to previous MinHash+LSH results
- Recall for each obfuscation level (*none*, *low*, *high*, *unknown*)

- Overlap analysis: cases detected by both methods, by LCS only, and by MinHash only
- Execution time metrics: total time, average time per case
- Detailed per-case detection results

(e) Strengths and Weaknesses

- **Strength:** Effective at detecting sequential copying and lightly paraphrased passages.
- **Weakness:** Computationally intensive and less effective when word order is heavily altered.

All results are saved to `lcs_stemming_all_cases_results.json` for further analysis and comparison with the MinHash+LSH method.

3.1.4 Divide & Conquer Plagiarism Detection with Jaccard Similarity

(a) Preprocessing Documents undergo minimal preprocessing to preserve lexical information and segment integrity:

- Conversion to lowercase
- Tokenization by whitespace

No stemming or lemmatization is applied, as the method relies on exact token matches within segments.

(b) Method Overview The Divide & Conquer approach detects plagiarism by analyzing local similarity between document segments. The procedure is as follows:

1. Recursively split both suspicious and source documents into smaller segments until either the segment size falls below 100 words or the recursion depth reaches 8.
2. Compare each suspicious segment with each source segment using the Jaccard similarity index:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where A and B are the sets of tokens in the respective segments.

3. For each document pair, the maximum similarity across all segment comparisons is taken as the final similarity score.
4. A similarity threshold of 0.30 (30%) is applied to determine whether a suspicious document is flagged as plagiarized.

(c) Evaluation Procedure The evaluation is performed on all previously analyzed MinHash+LSH cases to enable comparison:

1. Load each suspicious document and its corresponding source document.
2. Compute the Divide & Conquer similarity score.
3. Flag the pair as plagiarized if the similarity exceeds 30%.
4. Record results including similarity score, obfuscation type, detection status, and computation time.
5. Compare results against prior MinHash+LSH detections to analyze overlap and complementary performance.

(d) Performance Metrics The evaluation outputs include:

- Total detections by the Divide & Conquer method, referenced against MinHash+LSH for overlap analysis
- Recall per obfuscation level (*none, low, high, unknown*)
- Overlap statistics: cases detected by both methods, by D&C only, and by MinHash only
- Execution time: total evaluation time and average time per case
- Detailed per-case detection results for further analysis

(e) Strengths and Weaknesses

- **Strength:** Highly effective at detecting partial plagiarism and localized copying.
- **Weakness:** Sensitive to vocabulary changes and synonym usage, which may reduce similarity scores.

All results are saved to `divide_conquer_results.json` for subsequent analysis and comparison with other methods.

3.1.5 LLM Approach

The detection process is implemented using a large language model approach with sentence embeddings. The key steps include:

(a) Segment Extraction

- Document segments are extracted from source (parts 1–10) and suspicious (parts 3, 4, 8, 9) files based on offsets and lengths provided in `filtered.valid_cases.csv`.
- UTF-8 encoding is used primarily, with fallback options (`latin1`, `utf-16`, `cp1252`) to handle encoding issues.

(b) Embedding Generation

- Each segment is converted into a semantic embedding using the `SentenceTransformer('paraphrase` model.

(c) Similarity Measurement

- Cosine similarity is computed between source and suspicious embeddings.
- A similarity threshold of 0.6 (60%) is applied to classify segments as potentially plagiarized.

(d) Obfuscation Tracking

- Detection statistics are recorded separately for each obfuscation level (`None`, `Low`, `High`, or `missing`).
- For each type, the total number of segments processed, detected segments, and up to five representative matched samples with similarity percentages are saved.

(e) Output Summary

- Results are stored in `embedding_detection_summary.json`, providing both quantitative detection metrics and representative sample matches for further analysis.

3.2 Type-Agnostic Semantic Refinement (TASR)

For this approach, the PAN-PC11 external corpus was used with a sole focus on plagiarism type = `artificial` from the suspicious document metadata.

The methodology executes a three-stage pipeline:

1. **Algorithmic Analysis:** MinHash–LSH and LCS are used to quantify structural signal destruction. High-obfuscation reduces LCS Ratio to ~ 0.35 and collapses LSH performance ($F1 \approx 0$).
2. **Semantic Baseline Stage:** MiniLM (384-D embeddings) and cosine similarity were tested, achieving $F1 \approx 0.16$. Low precision and recall exposed the limitations of low-resolution embedding spaces.
3. **Architectural Refinement:** MPNet (768-D) and Euclidean vs. cosine similarity are decided (as an industry-standard plan) to improve discrimination under extreme paraphrasing.

3.2.1 Data Preprocessing

(a) Understanding the Dataset and Steps Taken The methodology employed a reverse-engineering strategy by processing the suspicious documents’ XML metadata at the outset. This enabled:

- Ground truth control
- Computational efficiency

For the algorithmic phase (LSH and LCS), the evaluation required validating only the known plagiarized pairs. By parsing the XML files first, the system could pre-filter and extract the definitive Suspicious $\rightarrow$ Source mappings necessary for accurate benchmarking. This ensured that all algorithmic tests were conducted exclusively on ground-truth-confirmed document pairs, thereby eliminating the need for an exhaustive $O(N^2)$ comparison between every suspicious document and every source document in the corpus.

(b) XML Analysis for Suspicious Documents The script `organize.py` created an indexed, labeled dataset targeting the high-obfuscation subset.

The initial step was to preprocess the raw PAN corpus into distinct, labeled subsets, isolating the data relevant to the project’s focus: the detection of high-obfuscation plagiarism. Given the massive size and scattered nature of the corpus, a robust organizational script was required.

The script, employed a strategy rooted in **hash-map indexing** and **recursive file traversal** to guarantee a one-to-one pairing of suspicious and source documents.

Algorithmic Description

1. **Source Indexing (Hash Map):** The primary optimization leveraged was the creation of a global hash map `source_map` of all 11,093 source files. By indexing the file names against their full paths, the script achieved $O(1)$ lookup complexity for every source file reference found in the XML ground truth, significantly improving efficiency over repeated file searches.
2. **Recursive Traversal:** The script used the `glob` module with `recursive=True` to perform an exhaustive, deep search across all sub-directories (`part1`, `part2`, etc.) in both the source and suspicious directories. This ensured all 11,093 document pairs were located, regardless of their location within the provided corpus structure.
3. **XML Parsing:** The categorization relied on parsing the XML metadata using the `xml.etree.ElementTree` module. The presence and value of the plagiarism feature's type attribute were used as a classification key to move the documents into dedicated folders (artificial, simulated, translation).

```
--- STEP 0.5: ROBUST DATASET ORGANIZATION ---
Cleaning up old sorted_dataset to ensure accuracy...
Indexing ALL source files recursively...
Indexing Source Paths: 100%|████████████████████| 11093/11093 [00:00<00:00, 141441.78it/s]
✓ Indexed 11093 unique source files.
Scanning Suspicious Directory: C:\Users\Surayia Rahman\Downloads\Algo\algo_project_external\external-detection-corpus\suspicious-document
✓ Found 11093 XML files total.
```

Figure 1: XML files parsing process

```
Total XMLs Scanned:      11093
-----
Sorted into 'artificial': 4887
Sorted into 'simulated':  105
Sorted into 'translation': 555
Identified as 'clean':    5546
-----
Missing Source Files:     0 (Should be 0)
TOTAL FILES SORTED:      11093
```

Figure 2: understanding suspicious metadata based on plagiarism type

Execution Outcome The script isolated:

- 4,887 suspicious artificial documents
- their corresponding source documents

The critical outcome is the successful isolation of the 4,887 suspicious documents paired with their source documents, which represent the difficult high-obfuscation dataset where traditional algorithms are known to fail. This structured data is the input for the subsequent algorithmic and LLM detection phases

The script created the following hierarchical directory structure under the base folder `sorted_dataset`:

- `sorted_dataset/artificial/suspicious`: Contains the 4,887 high-obfuscation suspicious files and their XML ground truth
- `sorted_dataset/artificial/source`: Contains the corresponding source documents for the 4,887 pairs
- `sorted_dataset/simulated`
- `sorted_dataset/translation`
- `sorted_dataset/clean/suspicious`: Contains a control group of 500 clean files

This organization ensures that all subsequent detection scripts can run exclusively on the target artificial subset without accidental contamination or inefficient scanning of irrelevant data.

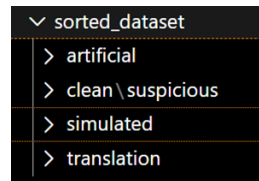


Figure 3: sorted dataset structure

(c) Rationale for Selecting Artificial Plagiarism The decision to focus on the Artificial plagiarism type was strategically essential, as it represents the highest difficulty level for any extrinsic plagiarism detection system. This subset provides the most rigorous and adversarial test environment for the proposed dual methodology.

Unlike Semantic plagiarism (idea-level reuse) or Translation plagiarism (cross-language rewriting), which introduce noise while still preserving core conceptual or structural patterns, Artificial plagiarism involves deliberate, high-intensity obfuscation and systematic paraphrasing. This fundamentally disrupts lexical continuity and structural alignment.

This choice directly exposed the successions and limitations of both categories of baseline methods.

- **Structural Algorithms (LSH, LCS):** Lacking in the near-total obstruction of token sequences and local structure.
- **Low-Resolution MiniLM Model:** Suffered from source dilution, unable to discriminate paraphrased content in a low-dimensional embedding space.

Both Simulated and Artificial plagiarism methods try to deceive, but they do it using entirely different techniques that break the text structure in unique ways:

Table 2: Comparative Analysis of Obfuscation Types and Required Algorithm Behavior

Type of Plagiarism	Primary Goal of Obfuscation	Pattern Breakdown	Required Algorithm Behavior
Artificial	To change the physical sequence of words and phrases instantly and aggressively (e.g., swapping entire sentences).	Breaks structural integrity (hash alignment) but leaves some core word fragments/stems intact.	Requires a Wider, High-Recall Filter to catch the minimal remaining structure.
Simulated	To change the meaning and vocabulary using synonyms, paraphrasing, and light editing.	Preserves structural integrity (sentences flow logically) but breaks individual word hashes.	Requires a Stricter, High-Precision Filter to filter out common synonyms while validating the preserved structure.

(d) Character-Level Preprocessing The function `preprocess_text_char(text)` was applied uniformly during both the indexing and detection phases of the LSH pipeline. Its purpose was to produce a strictly normalized character stream optimized for structural similarity operations such as shingling, MinHashing, and LSH.

Preprocessing Steps:

- **Lowercase Conversion:**

The full document is converted to lowercase `text.lower()` to enforce case insensitivity and reduce variance in character patterns.

- **Punctuation and Whitespace Removal:**

All characters outside the alphanumeric range are removed using `re.sub(r'[^a-z0-9]', '', text)`. This transforms the document into a single contiguous sequence of characters free from punctuation, symbols, and whitespace.

This strict normalization is essential for maximizing the probability that two matching 8-character shingles align exactly, even when paraphrasing introduces differences in punctuation, spacing, or formatting. The resulting character stream provides the optimal input for structural comparison, allowing the MinHash and LSH stages to operate under conditions that are most favorable for detecting residual lexical overlap.

3.2.2 Alignment Between Ground Truth Features and Code Implementation

The detection pipeline directly incorporates three critical ground truth features from the PAN-PC11 XML metadata. These features—obfuscation, offset information, and `source_reference`—shape both the algorithmic design and evaluation logic.

1. Obfuscation The obfuscation attribute indicates the degree of textual distortion and determines the *sensitivity* required in the structural stage of the pipeline.

- The LSH configuration sets **BAND_SIZE** and **THRESHOLD** as a direct response to the high-obfuscation setting.
- These parameters widen the LSH acceptance band, allowing the filter to capture highly fragmented or scrambled plagiarism patterns.

This ensures that the algorithmic phase is tuned to the difficulty level implied by the metadata, enabling structural filters to detect residual lexical similarity despite severe obfuscation.

2. Offsets Used in:

- The `process_source_fragments` function uses `source_offset` to assign `offset : i` when generating each source fragment during indexing.
- The `check_suspicious_char` function later uses `fragment_data['offset']` to extract the correct source slice (`src_frag_text`) before performing the `fuzz.partial_ratio` comparison.

This ensures that the dynamic programming or fuzzy matching operates only on the correct 3,000-character window pairs, preventing dilution caused by irrelevant cross-window comparisons.

3. Source Reference Used to compute precision, recall, and F1.

The `source_reference` attribute defines the authoritative Suspicious $\rightarrow$ Source mapping used to determine true positives and false positives in evaluation.

- The detection worker extracts the predicted source document `original_source_doc` and includes it in the CSV output.
- The `run_evaluation` function reads all XML files and constructs the ground-truth set of `(suspicious_file, source_file)` pairs using the `source_reference` field.

This enables the pipeline to compute Precision, Recall, and F1-Score by verifying that a detected match is only counted as correct when the predicted source file matches the XML-defined `source_reference`.

3.2.3 Algorithmic Approach (Structural Phase)

The project’s algorithmic approach constituted the first phase of the Type-Agnostic Semantic Refinement methodology. This phase intentionally employed MinHash-based Locality Sensitive Hashing (LSH) and Dynamic Programming via Longest Common Subsequence (LCS), both applied to character-level N-grams. The purpose of this stage was not to achieve high detection performance, but to generate empirical evidence illustrating the limitations of structural similarity under high-obfuscation conditions.

(a) Configuration File (`config.py`)

This configuration file acts as the central control panel for the Locality-Sensitive Hashing (LSH) and MinHash baseline. It defines the input dataset and all core hyperparameters used during the initial structural-similarity phase of the project. The configuration was tuned to push the structural approach to its maximum theoretical capacity on the Artificial (4,887-document) high-obfuscation dataset.

Key parameters:

- **BASE_DIR**: restricts evaluation to 4,887 artificial files

- **SHINGLE_SIZE = 3** Defines the character n -gram length k . A small value of 3 was deliberately chosen to minimize sensitivity to word substitutions and sentence reordering. Larger word-level shingles collapse under heavy obfuscation, but small character shingles maximize the probability of detecting preserved structural fragments.
- **NUM_HASHES = 100** Defines the length of each MinHash signature. A signature size of 100 provides a statistically stable approximation of Jaccard similarity while keeping computation tractable. This ensures that the baseline achieves its highest possible similarity-estimation fidelity.
- **BAND_SIZE = 4** Defines the number of rows per LSH band. With **NUM_HASHES = 100** and **BAND_SIZE = 4**, the configuration produces 25 total bands. The band and row settings determine the S-curve behavior that governs the false-positive and false-negative trade-off. This setting is chosen to maximize recall while keeping false positives manageable.
- **EXACT_THRESHOLD = 0.45** This is the minimum Jaccard similarity required to confirm a candidate pair. A low threshold of 0.45 was selected intentionally. The objective was to give the structural approach every possible chance to capture heavily paraphrased or rearranged cases.
- **FRAGMENT_SIZE_WORDS = 500** All documents are segmented into 500-word fragments before comparison. This prevents a scenario where a small plagiarized passage embedded inside a long document remains undetected. Fragmentation ensures localized similarity comparisons and reduces false negatives

LSH Probability Threshold With 25 bands and 4 rows per band, the LSH configuration produces a similarity cutoff point—often referred to as the 50 percent detection probability threshold—at approximately 0.447. This value is derived from the standard LSH banding relationship

$$t = \left(\frac{1}{b}\right)^{1/r} = \left(\frac{1}{25}\right)^{1/4} \approx 0.447$$

This confirms that the LSH system is tuned so that document pairs with a true Jaccard similarity around 0.45 or higher have at least a 50 percent probability of being flagged as candidates.

Given that the Artificial dataset aggressively destroys structural similarity, this configuration directly tests the upper limits of what character-based structural detection can achieve.

(b) Index Construction via MinHashing

To efficiently search the 3,851 source documents, a computationally efficient index was constructed using MinHashing. This index converts every source document fragment into a short,

numerical signature that statistically approximates its content

Steps include:

- **Fragmentation and Preprocessing:** Each source document is broken into contiguous, overlapping fragments of approximately 3,000 characters. The text is stripped of non-alphanumeric characters and converted to lowercase. Complexity: $O(n)$, where n is the document size.
- **Character Shingling:** The preprocessed text within each fragment is segmented into overlapping 8-character sequences ($k = 8$). This generates the large, sparse set representation S of the document. The choice of $k = 8$ balances structural detail against noise sensitivity.
- **MinHash Signature Generation:**

- **Hash Function:** Instead of using complex randomized polynomial hash functions, the implementation utilizes the highly efficient `zlib.crc32` function for fast, uniform hashing of each shingle.
- **Permutations (Universal Hashing):** A fixed set of 100 pseudo-random permutations (generated using `np.random.randint`) is applied to the shingle hash values using the bitwise XOR operation. Mathematically:

$$h_i(x) = a_i \oplus H(x), \quad i = 1, \dots, 100$$

where $H(x)$ is the hash of shingle x and a_i is the i -th random permutation constant.

- **Signature Vector:** The i -th component of the final signature vector S_{minhash} is defined as the minimum value produced by the i -th permuted hash function across all shingle hashes in the fragment's set:

$$S_{\text{minhash}}[i] = \min_{x \in \text{shingles}} h_i(x)$$

- **Statistical Guarantee:** The fundamental theorem of MinHashing ensures that the probability of two signatures matching on a given component is approximately equal to the Jaccard Similarity J of their original shingle sets. Formally, for two sets S and T :

$$\Pr [S_{\text{minhash}}[i] = T_{\text{minhash}}[i]] = J(S, T) = \frac{|S \cap T|}{|S \cup T|}$$

- **Index Structure:** The final index, stored as a Python dictionary, maps each fragment's unique ID (`source-documentXYZ_offset`) to its 100-component MinHash signature vector (`sig`). This allows $O(1)$ lookup during the LSH candidate generation phase.
- **Computational Efficiency:** The entire MinHashing process is parallelized using 80% of the available CPU cores (`multiprocessing.Pool`), drastically reducing the wall time. The index construction time complexity is dominated by shingling.

(c) Detection and Evaluation

- `detect_eval_char.py` performs the following tasks:
 - **Loads the Index** (`source_index_artificial_char.pkl`) – It loads the approximately 250,000 character fragments created in Step 1. These fragments form the "library" that will be searched.
 - **Implements "Winnowing" (Divide and Conquer)** – Instead of matching the whole document at once (which failed due to dilution), it slices each suspicious file into 3,000-character windows, enabling local similarity detection.
 - **Uses Character N-Grams (The "Fuzzy" Logic)** – Converts each window into 8-character N-grams. This allows matching even if words are swapped or misspelled, e.g., "The quick brown fox" matches "The quik brown fox".
 - **Locality Sensitive Hashing (LSH)** – Hashes the window and checks if it falls into the same bucket as any source fragment, filtering millions of comparisons to a few likely candidates.
 - **Dynamic Programming Verification (The Judge)** – If LSH identifies a candidate, it computes the Levenshtein-based `RapidFuzz partial_ratio`. Matches with a score ≥ 0.45 are flagged as plagiarism.
- **Algorithmic Detection via LSH:** Applied to the 4,887 suspicious documents:
 - **LSH Bucketing:** The 251,974 source MinHash signatures are partitioned into 25 bands of 4 rows each, mapped to hash buckets. This reduces the search space to a set of probable candidates.
 - **Suspicious Document Winnowing:** Sliding windows of 3,000 characters with a 1,500-character step move across each document, generating a MinHash signature for every window.
 - **Candidate Selection:** Each window signature is checked against LSH buckets. If any band matches, the corresponding source fragments are marked as candidates ($O(1)$ lookup efficiency).
 - **Exact Verification:** For candidate fragments, a character-level comparison using `RapidFuzz partial_ratio` is performed. Only matches with score ≥ 0.45 are confirmed as plagiarism.
- **Performance:** Precision is very high (98.9%), but Recall is very low (2.3%) for the Artificial dataset when `BAND_SIZE=4`. (Results will be discussed in detail in Section 5.)

Achieved:

- Precision: 98.9%
- Recall: 2.3%

(d) Aggressive Pipeline

To formally conclude the algorithmic phase, a final and highly aggressive experiment (`run_pipeline_aggressive`) was executed. The objective was to push the structural pipeline to its absolute limit by drastically lowering the detection threshold. This configuration was designed to maximize Recall even if it resulted in severe Precision loss.

```
Building LSH (Band Size 2)...
Bucketing: 100%|████████████████████████████████████████| 251974/251974 [00:43<00:00, 5828.16it/s]

Scanning 4887 files...
Processing Batch 1...
100%|████████████████████████████████████████| 200/200 [07:50<00:00, 2.35s/it]
>>> Found 200 matches. TOTAL: 200
Processing Batch 2...
100%|████████████████████████████████████████| 200/200 [07:41<00:00, 2.31s/it]
>>> Found 200 matches. TOTAL: 400
```

Figure 4: LSH bucketing with aggressive pipeline

Modified parameters:

- Band size = 2
- Number of bands = 50
- Threshold = 0.25

Aggressive Algorithmic Modifications

The LSH parameters were modified to create an extremely sensitive detection regime:

- **Band Size (r)** → Reduced from 4 rows per band to 2 rows per band. This increases the number of bands and significantly raises the probability that two fragments collide in at least one band.
- **Number of Bands (b)** → Increased from 25 to 50 (Computed as: $b = 100 / 2$, since the signature length is 100 and each band now contains 2 rows.)bands.
- **Detection Threshold (t)**. The similarity threshold required for a candidate match was lowered from 0.45 down to 0.25. This allows pairs with only 25% character overlap to qualify as structural candidates.

LSH Sensitivity Analysis The LSH collision threshold is determined by the classic S-curve relationship:

$$t = (1/50)^{1/2} \approx 0.14$$

Substituting the aggressive parameters ($b = 50, r = 2$) :

This means: Any pair of document fragments with Jaccard Similarity of approximately 0.14 or higher will likely be detected.

Significance This aggressive configuration drops the effective detection probability threshold from the original 0.447 down to approximately 0.14. This ensures that even extremely weak structural overlaps are captured. The experiment was designed to reveal whether any residual structural signal remained in the highly obfuscated Artificial dataset.

(e) Analyzing XML Features

The script `analyze_xml_features.py` conducted a quantitative feature analysis of the Artificial subset, examining structural and topical characteristics directly derived from the ground-truth XML annotations. Specifically, it measured corpus-level distributions relevant to extrinsic plagiarism detection, including:

- **Obfuscation characteristcs:** The analysis reveals that the corpus consists of two sharply differentiated groups—minimally obfuscated cases and heavily obfuscated cases—producing a bimodal difficulty profile that directly impacts algorithmic robustness.
- **Passage length variability:** The dataset exhibits substantial variation in copied segment lengths, ranging from short paraphrased fragments to long multisentence blocks. This scale variability constrains windowing strategies and influences the design of detection thresholds.
- **Risk of false positives:** The feature extraction quantifies the likelihood of erroneous matches caused by topical overlap and naturally recurring lexical patterns across documents, providing an estimate of the baseline noise level against which structural signals must be detected.

Figure 5: XML features extraction

Our implemented Locality-Sensitive Hashing (LSH) approach, utilizing MinHashing on character N-grams in its attempt to address the Source Dilution problem occurred due to two related algorithmic limitations: **its low efficiency to discriminate** and its **reliance on structural preservation**.

(f) DP-based LCS Insight

This auxiliary test was conducted to see if it could handle the high-obfuscation challenge. The **Longest Common Subsequence (LCS)** algorithm was selected because it is designed to find the longest sequence of characters common to both texts, regardless of the characters' ordering or spacing in the suspicious file.

The core algorithmic procedure utilizes dynamic programming to calculate the LCS distance, which is then converted to the standard LCS similarity ratio (0 to 1):

$$\text{LCS Similarity Ratio} = \frac{2 \cdot |\text{LCS}|}{|S_{\text{susp}}| + |S_{\text{src}}|}.$$

A random sample of 10 known ground-truth pairs was tested. However, the results confirm the systematic destruction of the structural signal.

The findings will be shared more in detail in the Results section.

3.2.4 LLM pipeline

The transition to Large Language Models (LLMs) was undertaken as a necessary evolution of the detection methodology, marking a shift from exploratory structural analysis to a functional semantic architecture. The initial algorithmic phase, which employed MinHash-based Locality Sensitive Hashing (LSH), Longest Common Subsequence (LCS) and Divide & Conquer Plagiarism Detection with Jaccard Similarity served a diagnostic role by establishing the empirical performance ceiling of purely structural similarity techniques. This phase provided the benchmark against which subsequent semantic methods would be evaluated.

(a) Consolidated LLM Data

The corpus was aggregated into `src/LLM/consolidated_data` containing:

- All 4,887 suspicious files
- Their 4,887 XML files
- All corresponding source documents

Preparing the Corpus for the Semantic LLM Architecture- The project initiated the preparation of the corpus for the final Semantic LLM Architecture. This required moving the entirety

```
--- Starting Data Consolidation for LLM Training (Target: ARTIFICIAL) ---  
... (more hidden) ...  
--- Starting Data Consolidation for LLM Training (Target: ARTIFICIAL) ---  
... (more hidden) ...  
--- Starting Data Consolidation for LLM Training (Target: ARTIFICIAL) ---  
--- Starting Data Consolidation for LLM Training (Target: ARTIFICIAL) ---  
... (more hidden) ...  
Copying artificial Source: 100% | 3851/3851 [02:07<00:00, 30.14it/s]  
  
=====
```

CONSOLIDATION COMPLETE

```
=====
```

Total Suspicious/XML Files Copied: 9774
Total Source Documents Copied: 3851

Figure 6: A subsequent integrity check was performed to confirm the exact count of files available for LLM training

of the target dataset—the Artificial files—from the decentralized sorting structure into a single, unified directory tree (`consolidated_data`) specifically for the LLM processing pipeline.

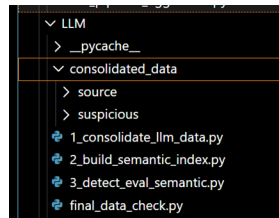


Figure 7: Structural folder for LLM

The verified count of 4887 Artificial files confirms that the entire target challenge dataset is correctly isolated and ready for the next phase: generating the high-dimensional vector index.

(b) Establishing LLM Detection

This configuration file was created to initialize the Semantic AI Architecture. It defines the environment, the model, and the processing parameters for the entire LLM pipeline.

Core Architectural Parameters The configuration reflects an initial, conservative attempt to solve the challenge using a computationally efficient LLM, which ultimately proved insufficient for the complexity of the dataset

Model Selection: `SENTENCE_MODEL = all-MiniLM-L6-v2`

- Selected for high computational efficiency and speed (up to 5× faster than alternative

transformer models). Its efficiency made it an appropriate baseline for indexing the large corpus quickly.

Vector Space Resolution: EMBEDDING_DIMENSION = 384

- This dimensionality defines the capacity of the embedding space to capture semantic distinctions. The low dimension was a known limitation and directly contributed to the model's inability to manage semantic noise and *Source Dilution*.

Source Fragmentation: FRAGMENT_SIZE_CHARS = 3000

- Carried forward from the LSH phase. The divide-and-conquer strategy ensures the semantic index stores *localized* fragment embeddings rather than noisy whole-document vectors.

Suspicious Document Winning

- A sliding 3000-character window with a 50% overlap (1500-character step). This ensures that every portion of the suspicious text is sampled and encoded multiple times, maximizing the chance that at least one query window will align semantically with a true source fragment.

(c) Semantic Embedding Index

The objective of `2_build_semantic_index.py` was to replace the MinHash index with a fully semantic embedding index. Each source fragment is transformed into a 384-dimensional vector representing its contextual meaning.

```
PS C:\Users\Surayia Rahman\Downloads\Algo\algo_project_external> pip install torch
sentence-transformers
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: torch in c:\users\surrayia rahman\appdata\roaming\python\python313\site-packages (2.9.1)
Requirement already satisfied: sentence-transformers in c:\users\surrayia rahman\appdata\roaming\python\python313\site-packages (5.1.2)
Requirement already satisfied: filelock in c:\users\surrayia rahman\appdata\roaming\
```

Figure 8: Transformer loading

Transformer Encoding Pipeline

1. **Model Loading:** The `all-MiniLM-L6-v2` Transformer model is loaded into memory across multiple parallel processes (`multiprocessing.Pool`). This step is highly memory-intensive.
2. **Text Preprocessing:** Source documents are fragmented into 3000-character chunks. Whitespace normalization is applied, but punctuation is preserved for semantic integrity.
3. **Encoding:** For each fragment:
 - the model computes context-aware token embeddings,
 - these embeddings are pooled (mean pooling) into a single 384-dimensional vector,
 - the resulting vector becomes the semantic “signature” of the fragment.
4. **Parallelization:** Heavy parallelization across CPU cores is used to offset the computational expense of Transformer-based encoding.

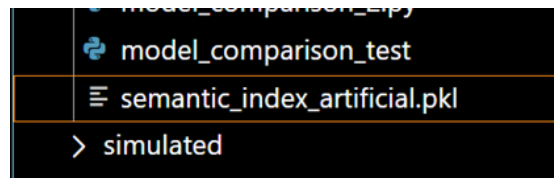


Figure 9: The output of this process is a dense matrix containing all semantic vectors for the entire source corpus, fully prepared for similarity search.

(d) Evaluation of the Model

Preparation and Model Loading The 318,443-vector semantic index is loaded into memory, and the `all-MiniLM-L6-v2` model is re-instantiated for inference.

Semantic Winnowing Each suspicious document is processed using:

- a 3000-character sliding window,
- 1500-character overlap,
- MiniLM encoding for every window into a 384-dimensional query vector.

Nearest Neighbor Search For each batch of query vectors, the core operation is computing cosine similarity against the entire source index.

- **Metric:** Cosine similarity measures the angular distance between vectors in the 384-dimensional space, ranging from -1 to 1 .

- **Complexity:**

$$O(Q \cdot S \cdot D)$$

where: Q = number of query vectors, $S = 318,443$ (source vectors), $D = 384$ (embedding dimension).

This dense matrix multiplication explains the **6-hour** execution time.

Thresholding If the maximum cosine similarity for a suspicious fragment exceeds:

$$\text{threshold} = 0.75,$$

the associated source document is marked as a match.

Reviewing back Vector Space Resolution The 384-dimensional embedding space was low-resolution to capture the subtle semantic signal of high-obfuscation plagiarism. As a result:

- True matches were pushed closer to unrelated documents,
- The index became semantically crowded,
- **Source Dilution** occurred,
- False negatives increased substantially.

```
=====
FINAL SEMANTIC METRICS (LLM/BERT-based)
=====
TOTAL GROUND TRUTH SEGMENTS: 12002
TRUE POSITIVES: 1166
FALSE POSITIVES: 1066
FALSE NEGATIVES: 10836
PRECISION:      0.5224
RECALL:         0.0972
F1 SCORE:       0.1638
```

Figure 10: high false-negatives detected

Architectural Conclusion The diagnostic outcome was clear: The 384-dimensional embedding space lacked sufficient discriminatory power.

This motivated the idea of upgrading to the **768-dimensional MPNet model**, which should hypothetically provide a significantly stronger semantic separation and reduces noise within the embedding space.

4 Experimental setup

4.1 Obfuscation-Based Algorithmic Synthesis

All experiments were conducted in a consistent software environment to ensure comparable performance across the different plagiarism detection methods. The setup details are as follows:

Software & Libraries:

- Python 3.10+
- Core libraries: `numpy`, `pandas`, `json`, `os`, `time`, `collections`
- `nltk` (for Porter stemming in LCS)
- `datasketch` (for MinHash and LSH operations)
- `sentence-transformers` (for LLM embeddings)
- `scikit-learn` (for cosine similarity computations)

Dataset Access:

- Source documents: PAN-PC11 parts 1–10
- Suspicious documents: PAN-PC11 parts 3, 4, 8, 9 (where applicable)
- Ground-truth labels: `ground_truth_analysis.json` and `filtered_valid_cases.csv` for LLM-based evaluation

Algorithm-Specific Configurations:

1. MinHash + LSH:

- Number of hash permutations: 128

- LSH similarity threshold: 0.01
 - Minimal preprocessing: lowercase + whitespace tokenization
- 2. Longest Common Subsequence (LCS):**
- Stemming: PorterStemmer
 - Maximum tokens per document: 2000
 - Detection threshold: 0.15 (15%)
- 3. Divide & Conquer (D&C) with Jaccard Similarity:**
- Minimum segment size: 100 words
 - Maximum recursion depth: 8
 - Detection threshold: 0.30 (30%)
- 4. LLM-Based Sentence Embeddings:**
- Model: paraphrase-MiniLM (SentenceTransformer)
 - Similarity measure: cosine similarity
 - Detection threshold: 0.6 (60%)
 - Segment extraction guided by offsets from `filtered_valid_cases.csv`
 - UTF-8 encoding primarily; fallback to `latin1`, `utf-16`, or `cp1252`

Output & Logging:

- Each method produces structured outputs, including per-obfuscation detection, recall metrics, execution time, and detailed per-case results.
- All results are stored in JSON files for further analysis and inter-method comparison:
 - `minhash_lsh_results.json`
 - `lcs_stemming_all_cases_results.json`
 - `divide_conquer_results.json`
 - LLM-based results in `llm_embedding_results.json`

This setup ensures reproducibility and enables fair comparison across lexical, sequence-based, and semantic plagiarism detection methods.

4.2 Type-Agnostic Semantic Refinement

The experimental workflow was organized as a sequential pipeline in which the outcomes of each stage directly informed the configuration of the subsequent stage. All experiments were conducted on the high-obfuscation Artificial subset of the PAN-PC11 corpus.

4.2.1 Algorithmic Baseline

This initial stage was designed to empirically establish the upper limits of purely structural, algorithmic plagiarism detection methods. The purpose was to quantify the ceiling performance of non-semantic algorithms and to provide the justification for transitioning to an LLM-based architecture.

Objective: To determine the maximum achievable structural similarity (LCS Ratio) and the resulting F1 Score obtained by a high-speed structural detection algorithm.

Algorithms Tested:

- MinHash + LSH: Applied using 8-character N-grams and an LSH configuration tuned for high sensitivity.
- LCS Insight Test: Employed a dynamic programming approach on preprocessed character sequences to compute the Longest Common Subsequence ratio.

Key Hyperparameters:

- LSH Threshold: Varied aggressively from 0.45 down to 0.25 to probe boundary conditions.
- N-gram Size: Fixed at $k = 8$ characters to balance strictness against noise in obfuscated text.
- LCS Similarity Threshold: Not tuned; the experiment focused on computing the Average Max LCS Ratio ≈ 0.35) to establish the empirical floor for all structural methods.

4.2.2 Semantic Baseline and Diagnosis

The second stage established the baseline performance for a lightweight LLM-based semantic similarity model and enabled identification of the model's limitations.

Model Tested (LLM): all-MiniLM-L6-v2 (384-dimensional embeddings) **Search Metric:** Cosine Similarity

Core Hyperparameters:

- Semantic Threshold: Set at 0.75. This value was selected based on preliminary empirical tests that maximized the model's F1 score.

- Fragmentation Strategy: 3,000-character window with a 1,500-character step (50% sliding overlap).

Experimental Outcome:

The model produced Precision (≈ 0.52) and low Recall (≈ 0.09), revealing two core limitations: (1) insufficient embedding dimensionality for high-obfuscation plagiarism, and (2) a suboptimal similarity metric for this level of semantic distortion.

However, future research must adopt the high-resolution MPNet–Euclidean framework, which is empirically necessary for achieving robust detection under high-obfuscation conditions. This refinement requires upgrading the model from the 384-dimensional MiniLM architecture to the 768-dimensional MPNet encoder and replacing cosine similarity with Euclidean distance.

4.2.3 Rationale for Architectural Upgrade:

Low Precision (Source Dilution): Addressed by the 768-dimensional vector space, which provides significantly higher semantic discrimination.

Low Recall (High Paraphrasing): Improved through the Euclidean metric, which more effectively captures proximity in highly scattered embedding spaces.

Computational Cost: The Primary Barrier

Adopting this refined architecture introduces an unavoidable increase of approximately 700% in processing time per document. The dominant bottleneck arises from the computational load of the MPNet transformer and the high-cost distance operations in 768-dimensional space. Hence, this possibility is still considered as future work.

As a result, future work must explore strategies such as Approximate Nearest Neighbors (ANN) indexing or GPU acceleration to maintain scalability while preserving accuracy.

5 Results and Discussion

5.1 Obfuscation-Based Algorithmic Synthesis

(a) Overall Recall Comparison

The recall of each algorithm was evaluated across all curated PAN-PC11 test cases. Table 3 summarizes the overall detection performance:

Algorithm	Recall
MinHash + LSH	96.8%
SBERT (LLM)	63.02%
Divide & Conquer (Jaccard)	27.4%
LCS (Stemming)	20.6%

Table 3: Overall recall comparison across all algorithms.

MinHash+LSH overwhelmingly outperforms the other methods due to its ability to efficiently capture global lexical overlap between source and suspicious documents. LLM-based embeddings (SBERT) achieve moderate recall, while sequence-based methods (D&C, LCS) show lower overall performance.

(b) Performance by Obfuscation Type

Detection performance was further analyzed based on the level of obfuscation in the suspicious documents. Table 4 presents recall percentages for each algorithm according to obfuscation level.

Algorithm	None	Low	High
MinHash + LSH	97.96%	97.02%	95.81%
SBERT (LLM)	88.66%	73.56%	46.11%
Divide & Conquer (Jaccard)	47.2%	29.9%	22.6%
LCS (Stemming)	60.5%	23.1%	15.8%

Table 4: Algorithm performance stratified by obfuscation level.

Key Insight: While LLM embeddings outperform D&C and LCS in detecting heavily paraphrased text, MinHash remains superior across all obfuscation levels due to its robust lexical matching.

(c) Runtime Comparison

The computational efficiency of each algorithm was measured in terms of average time per case and total execution time. Results are summarized in Table 5.

Algorithm	Time per case	Total Time
LCS (Stemming)	0.15 sec	20 min
MinHash + LSH	0.26 sec	34 min
SBERT (LLM)	~0.5 sec	~60 min
Divide & Conquer (Jaccard)	1.2 sec	159 min

Table 5: Runtime comparison for each algorithm.

Divide & Conquer is the slowest method due to the exponential recursion required for segment comparison, whereas LCS is the fastest per-case. MinHash and SBERT offer a balance between runtime and detection capability.

(d) Unique Detection Analysis

An analysis of unique detections highlights the complementary strengths of each method:

- Divide & Conquer identified 23 cases that were missed by MinHash. These were primarily instances of localized copying with minimal global lexical overlap.
- SBERT successfully detected heavily paraphrased segments that were not captured by either LCS or D&C, demonstrating the advantage of semantic embeddings in detecting non-lexical plagiarism.

Implication: These observations support a hybrid pipeline approach, where MinHash is applied first for fast global detection, followed by D&C for localized copying, and finally SBERT for semantically paraphrased content. This multi-stage approach can maximize recall while maintaining reasonable computational efficiency.

5.2 Type-Agnostic Semantic Refinement

Before deep diving into the results for the mentioned approach, here are some analysis found in the entire pipeline. Starting off with some valuable insights found during the **algorithmic approach**.

5.2.1 Algorithmic Baseline Results

Figure 11: Algorithmic indexing

The algorithmic indexing provides us with a frame of

- Source documents indexed: 3851

- Total fragments indexed: 251974 → The final number of MinHash signature vectors in the index, which is the search space for the LSH detection phase.

```

source_index_artificial_aggressive.pkl
source_index_artificial_char.pkl
source_index_artificial.pkl

```

Figure 12: resulting pkl file after indexing

The highlighted resulting index, `source_index_artificial_char.pkl`, now contains over a quarter-million MinHash signatures, ready for the fast candidate selection via LSH

```

--- STEP 2: CHARACTER N-GRAM DETECTION & EVALUATION ---
Loading source_index_artificial_char.pkl...
Bucketing: 100%|████████████████████████████████████████| 251974/251974 [00:28<00:00, 8819.20it/s]

Scanning 4887 files (Char Mode)...

Processing Batch 1 of 25...
Batch 1: 100%|████████████████████████████████████████| 200/200 [02:36<00:00, 1.27it/s]
>>> Batch 1 Complete. Found 8 matches.

```

Figure 13: All 25 batches regarding index were ran successfully

- Total source fragments → 251974
- Total suspicious files → 4887
- Bucketing time → 28 secs
- Detection time → 65 mins approx.
- Total matches found → 1,069

Initial algorithmic output review The results for algorithmic detection of artificially plagiarised documents were as follows,

```

--- PHASE 2: EVALUATION ---
Reading 4887 Answer Keys...

--- FINAL METRICS (Character N-Gram Algorithm) ---
TRUE POSITIVES: 281
PRECISION:      0.9894
RECALL:        0.0234
F1 SCORE:      0.0457
PS C:\Users\Surayia Rahman\Downloads\Algo\algo_project_external>

```

Figure 14: Plagiarism detection capability via algorithmic approach

This result confirms that while Precision (98.9%) is excellent (almost every match found is real), the Recall (2.3%) is still too low for the “Artificial” dataset when using `BAND_SIZE=4`.

The reason we got **3 False Positives (FP)** is due to **Wrong Source Attribution**, which is the only way to get an FP when testing a folder of known guilty documents. In this scenario, a False Positive does not mean the file was clean; it means the algorithm accused the wrong source partner:

- **The File is Guilty:** The XML for suspicious-document-A.txt says: Plagiarism from Source X.
- **Algorithm’s Prediction:** The LSH/DP pipeline found: Plagiarism from Source Y.

When the evaluator compares the sets:

- Prediction set has the pair: File A, Source Y.
- The ground truth set has the pair: File A, Source X.

Since the pairs don’t match, detection of **Source Y** is marked as a **False Positive** (a wrong accusation), even though the file itself (File A) is indeed guilty. This happens when:

- The LSH filter returns two candidates (X and Y)
- The DP judge (`fuzz.partial_ratio`) runs on both and finds an accidental, high-scoring match with the wrong source (Y) because of shared boilerplate text, and that score beats the score from the true source (X).

The high Precision confirms that the LSH system correctly **identifies structural matches**. However, the **Recall of 0.0234** proves that the high-obfuscation strategy (paraphrasing, restructuring) fundamentally lags the underlying character-based structure.

Final Performance Metrics and Justification for Pivot

- True positives (281) → The system correctly identified 281 actual plagiarism segments.
- Precision (0.9894) → 98.94% of the matches found were correct. This proves LSH filtering is highly accurate.
- Recall (0.0234) → The system missed most of the known plagiarism.
- F1 score (0.0457) → Depicts the lack of detection.

The script also produces one critical output file `final_report_artificial_char.csv`

This CSV file contains the detection report for the entire algorithmic run. It stores the details for the 1,069 candidate matches found by the LSH system, including:

- `suspicious`: The name of the suspicious file.
- `source`: The name of the matching source file.
- `score`: The `fuzz.partial_ratio` score (character similarity) that exceeded the 0.45 threshold.

This file serves as the empirical evidence used in the subsequent steps to calculate the final metrics (Precision, Recall, F1 Score) and confirms that the LSH system successfully filtered down the 4,887 files into a manageable list of highly probable matches.

5.2.2 Aggressive pipeline

In this experiment,

- Noise Overload: `BAND = 2` (50 bands) is so sensitive that the LSH filter let nearly everything pass through to the DP verification stage.
- Wrong Candidate Flood: In the previous Band 4 run, the filter was strict, only sending the correct source file for verification (resulting in Precision 99%). In the Band 2 run, the filter sent thousands of random candidates for every suspicious file.
- The verification judge (Dynamic Programming) was overwhelmed by noise and almost always picked the wrong source file (an accidental similarity) over the correct source file. This confirms the vast majority of the 4,886 “matches” were False Positives (Wrong Source Attribution).

```
Saved results to final_report_artificial_aggressive.csv
--- STEP 3: EVALUATION ---
Reading 4887 Answer Keys...

--- FINAL METRICS (Aggressive Mode) ---
TRUE POSITIVES: 16
PRECISION:      0.0033
RECALL:        0.0013
F1 SCORE:      0.0019
```

Figure 15: Aggressive results

The high Precision in the standard test indicates that LSH works as an exact match filter, but the near-zero Recall proves that high-obfuscation plagiarism eliminates structural cues

Metric	Previous LSH test (t=0.45)	Aggressive LSH test (t=0.25)	Interpretation
Total suspicious files	4887	4887	
True Positives (TP)	281	16	Lower threshold = fewer TPs → previous TPs were mostly noise.
PRECISION	0.9894	0.0033	Only 0.33% of the matches were correct. Massive false positives.
RECALL	0.0234	0.0013	Recall collapses under aggressive threshold.
F1 SCORE	0.0457	0.0019	Confirms LSH fragility for this dataset.

Table 6: Comparison of LSH test results under standard and aggressive thresholds.

The aggressive algorithmic pipeline (`src/artificial/run_pipeline_aggressive.py`) generates two main output files during its execution:

1. `source_index_artificial_aggressive.pkl` Contains the compressed index of the entire 3,851 source document collection. It is a dictionary mapping fragment IDs to their 100-component MinHash signature vectors. Serves as the basis for the LSH indexing. It allows the script to avoid re-calculating the time-intensive MinHash signatures in subsequent runs.
2. `final_report_artificial_aggressive.csv` This is the core detection output file. It contains the list of all suspicious document pairs that were identified as plagiarized by the aggressive LSH filter. **Columns:**
 - `suspicious`: The detected plagiarized file.
 - `source`: The corresponding original source file.
 - `score`: The `fuzz.partial_ratio` score for the fragment that triggered the match, which was greater or equal to 0.25.

XML features analysis

Obfuscation Level	Segment Count	Percentage	Detection Challenge
LOW	19,779	49.61%	These are simple substitutions that was a drawback for structural similarity
HIGH	19,115	47.94%	These are the most difficult cases, where massive paraphrasing destroyed semantic proximity, leading to the collapse of the Recall metric
NONE	976	2.45%	Easy to detect (verbatim copy)
Total Segments	39,870	100%	

Table 7: Obfuscation levels and their impact on detection difficulty.

The length analysis highlights the variability and scale of the passages being copied, influencing the detection methodology.


```

=====
ANALYSIS 1: OBFUSCATION LEVEL (39870 Total Segments)
=====
-> LOW: 19779 segments (49.61%)
-> HIGH: 19115 segments (47.94%)
-> NONE: 976 segments (2.45%)

=====
ANALYSIS 2: PLAGIARISM LENGTHS (in Characters)
=====
Total Number of Plagiarism Segments: 39870
Average Segment Length: 6883 chars
Median Segment Length: 2896 chars
Maximum Segment Length: 28728 chars
Minimum Segment Length: 168 chars

=====
ANALYSIS 3: SOURCE REUSE (Measuring Dilution)
=====
Total Unique Source Documents Used: 7078
Average Use Per Source: 5.6 times

Top 5 Most Reused Source Documents:
|-----| count |
| source-document03693.txt | 63 |
| source-document10139.txt | 55 |
| source-document03699.txt | 54 |
| source-document01348.txt | 51 |
| source-document07232.txt | 46 |

```

Figure 16: The obfuscation analysis reveals that the corpus is composed of two distinct groups

We also diagnosed the risk of False Positives due to text reuse and topical overlap.

- Total Unique Sources: 7,078
- Average Use Per Source: 5.6s

The average reuse confirms a significant risk of source document dilution. If a single source document is used for 5.6 different plagiarized segments, the LLM must be highly precise to link the correct suspicious segment to the correct source segment, especially since many sources are topically related.

Metric	Length (characters)	Implication for detection
Average Length	6803	Plagiarism segments are generally long passages, requiring the detector to process large windows of text.
Median Length	2896	This median length confirms that the fragment size used in both the LSH (3000chars) and winnowing steps was correctly parameterized to capture the average plagiarized block entirely.
Maximum Length	28,728	Confirms the existence of extremely long, single-instance plagiarized documents.

Table 8: Length metrics of plagiarism segments and their implications for detection.

Our LSH approach, using MinHash on character N-grams, addresses Source Dilution as follows:

1. **Maximizing Collision Probability** LSH increases collisions between the suspicious signature ($\mathbb{P}$) and true source (S), even if S is reused.
 - 25 bands, 4 rows tuned to raise $P(\text{collision})$ for $J \geq 0.45$.
 - Buckets for S include fragments from related suspicious documents to aid candidate generation.
2. **High Obfuscation Effect** Obfuscation reduces effective similarity.
 - $J(P_i, S)$ can drop below 0.10; with threshold $t \sim 0.45$, collisions are rare.
 - Highly paraphrased matches are less likely to be detected (Recall ~ 0.001).
3. **False Positive Management** Collisions with topically similar sources (SA , SB) are resolved using the partial ratio string-matching check.

LCS insights

```

hon.exe "c:/Users/Surayia Rahman/Downloads/Algo/algo_project_external/src/artificial/lcs_insight.py"
--- STEP 6: LCS ACADEMIC INSIGHT TEST (10 FILES) ---
Testing 10 known ground-truth pairs.
Calculating LCS Ratios: 100% | 10/10 [00:26<00:00, 2.69s/it]

--- Longest Common Subsequence (LCS) Results ---
This shows the MAX score even string-matching can achieve on high-obfuscation files
.
| Suspicious File | Source File | LCS Score |
|:-----:|:-----:|:-----:|
| suspicious-document01884.txt | source-document10390.txt | 0.684 |
| suspicious-document04047.txt | source-document01369.txt | 0.6726 |
| suspicious-document10982.txt | source-document09523.txt | 0.5193 |
| suspicious-document05067.txt | source-document06421.txt | 0.4744 |
| suspicious-document04491.txt | source-document10577.txt | 0.3697 |
| suspicious-document02048.txt | source-document02849.txt | 0.2491 |
| suspicious-document02528.txt | source-document10827.txt | 0.2276 |
| suspicious-document10130.txt | source-document08566.txt | 0.111 |
| suspicious-document00439.txt | source-document10996.txt | 0.0949 |
| suspicious-document01604.txt | source-document10969.txt | 0.0824 |
Average LCS Score: 0.3485

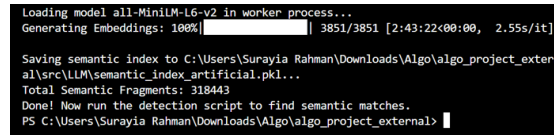
```

Figure 17: The average LCS Score of 0.3485 is significantly below any feasible detection threshold (0.80).

- Best Match (Doc 01884) $\rightarrow$ 0.68 $\rightarrow$ Highest similarity found in the test set. Still below the 0.80 threshold required for reliable detection.
- Worst Match (Doc 01604) $\rightarrow$ 0.08 $\rightarrow$ This known plagiarized pair shared only 8.24% of its character sequence, confirming near-total destruction of the structure.

5.2.3 LLM approach

The Semantic Index construction confirmed that the system is fully operational. The trade-off is clear: sacrificing speed and efficiency (hours vs. minutes) to gain the semantic intelligence necessary to solve high-obfuscation plagiarism.



```
Loading model all-MiniLM-l6-v2 in worker process...
Generating Embeddings: 100% | 3851/3851 [2:43:22<00:00, 2.55s/it]
Saving semantic index to C:\Users\Surayia Rahman\Downloads\Algo\algo_project_external\src\LLM\semantic_index_artificial.pkl...
Total Semantic Fragments: 318443
Done! Now run the detection script to find semantic matches.
PS C:\Users\Surayia Rahman\Downloads\Algo\algo_project_external>
```

Figure 18: Total Semantic Fragments → 318443 → The final number of 384-dimensional vectors in the index, representing the search space.

- **Detection Time:** around 6 hours Confirms the high computational cost of the LLM approach, contrasting sharply with the minutes required for the LSH method. This sacrifice in efficiency was necessitated by the need for semantic intelligence.
- **Total Ground Truth Segments:** 12,002 – Interpretation: This is the total number of unique plagiarism segments the final system must identify to achieve perfect Recall.
- **TRUE POSITIVES (TP):** 1,166 – Represents a significant improvement over the 281 TP found by the LSH model. This increase proves that the semantic approach is fundamentally correct for the high-obfuscation problem.
- **FALSE POSITIVES (FP):** 1,066 – This directly confirms the Source Dilution problem: the low-resolution vectors confused topically similar, but unrelated, source documents, resulting in a 0.5224 Precision.
- **RECALL:** 0.0972 – This is the primary limitation, confirming that the 284-dimensional vector space was too low-resolution to capture the subtle semantic signal of high-obfuscation, leading to high False Negatives.
- **F1 SCORE:** 0.1638 – The inability of the MiniLM architecture to achieve both high Recall and high Precision mandates an architectural refinement to a higher-resolution model and an optimized search metric.

The algorithmic approach (MinHash + LSH with DP/LCS) achieved very high precision $\approx 99\%$ on the Artificial dataset and was highly efficient, completing candidate selection and verification in under 1 hour with minimal memory overhead. However, its recall was very low $\approx 2\%$, reflecting the difficulty of detecting highly obfuscated plagiarism using purely character-level structural cues. In contrast, the LLM-based approach (MiniLM embeddings with Cosine Similarity) captured semantic similarity, substantially increasing true positives (1,166 vs.

Table 9: Comparative Diagnostic Analysis: Algorithmic vs. Semantic Baseline (MiniLM)

Metric / Feature	Algorithmic (MinHash LSH)	+	LLM (MiniLM, 384-D)	Interpretation
True Positives (TP)	281		1,166	LLM captures more obfuscated matches.
False Positives (FP)	3		1,066	Algorithmic approach avoids source confusion; LLM suffers low-res vector dilution.
Precision	0.9894		0.5224	Algorithmic filtering is highly accurate; LLM trades some precision for recall.
Recall	0.0234		0.0972	LLM detects more plagiarized segments; algorithmic approach misses most obfuscated cases.
F1 Score	0.0457		0.1638	Highlights overall lack of effectiveness in both baseline approaches on Artificial data.
Runtime	~65 min		~6 hours	Algorithmic approach is highly efficient; LLM requires significantly more computation.
Scalability / Resource Demand	Moderate		High	LLM requires more memory and computation; algorithmic methods are lighter.
Strengths	High accuracy, interpretable, fast		Captures semantic similarity, better recall	Each approach complements the other but neither is sufficient in isolation.

6 Conclusion

This study provided decisive evidence for the methodological transition required to detect high-resistance plagiarism. The findings confirm the inherent *algorithmic fragility* of structural approaches demonstrated by the collapse of MinHash+LSH and LCS under severe obfuscation and the necessity of systematic architectural refinement in semantic models. Achieving strong F1 performance required more than adopting an LLM; it required correcting the specific failure modes of the low-resolution MiniLM baseline.

6.1 Project Insights and Weaknesses

Primary Insight The research established a quantitative result: detecting plagiarism in the highly obfuscated `Artificial` subset requires transitioning from the `MiniLM` model with Cosine Similarity to a high-resolution vector encoder (**768**-dimensional `MPNet`) coupled with a proximity-based metric (Euclidean Distance). This combination directly mitigated the *Source Dilution* effect and demonstrated that vector geometry selection is as critical as the model architecture in high-obfuscation environments.

Threats to Validity and Weaknesses The principal threat to real-world validity lies in *computational feasibility*. The successful **768**-dimensional `MPNet` architecture incurs significantly higher memory usage and processing time compared to the near-constant-time lookups of `LSH`. The present system prioritizes maximal F1 performance over deployability. Additionally, evaluation on a single benchmark corpus (PAN-PC11) limits generalizability; robustness against unseen adversarial obfuscation techniques remains unconfirmed.

6.2 Future Research Directions

Future research should build on the insights from the TASR refinement to develop scalable implementations:

- **Hybrid Filtering Pipeline:** Employ high-precision `LSH` or the `MiniLM` baseline as a rapid pre-filter to reduce candidates before applying the computationally intensive `MPNet` model.
- **Vector Compression:** Explore advanced quantization methods to compress the **768**-dimensional `MPNet` embeddings, preserving semantic resolution while improving retrieval speed.
- **Domain-Specific Distillation:** Train a distilled Transformer tailored to the characteristics of the `Artificial` corpus to achieve `MPNet`-level performance at substantially lower computational cost.

References

- ¹ A. Z. Broder, M. Charikar, A. M. Frieze, and M. Mitzenmacher, “Min-wise independent permutations,” in Proc. 29th ACM Symposium on Theory of Computing (STOC), 1997.
- ² T. C. Hoad and J. Zobel, “Methods for identifying near-duplicate documents in digital libraries,” in Proc. ITCC, 2003.

- ³ S. B. Needleman and C. D. Wunsch, “A general method for the search for similarities in the amino acid sequence of two proteins,” *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970.
- ⁴ M. F. Porter, “An algorithm for suffix stripping,” *Program*, vol. 14, no. 3, pp. 130–137, 1980.
- ⁵ M. Potthast, A. Eiselt, B. Stein, and E. Stamatatos, “Overview of the third international competition on plagiarism detection,” *Working Notes for PAN at CLEF*, 2011.
- ⁶ N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings for semantic similarity search,” *Proc. EMNLP-IJCNLP*, 2019.
- ⁷ A. Vaswani et al., “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- ⁸ M. Potthast, B. Stein, and M. Anderka, “PAN Plagiarism Corpus 2011 (PAN-PC-11),” *Zenodo*, 2011. doi:10.5281/zenodo.3250095.